Jaime Manuel Dy 202316216
Maria Louise C. Quitoriano 202206275

CoE 163 - Software Exercise 1

1. ***Algorithm***:

For this SE, we were tasked to create a function that will be able to find the number of turns it takes to unlock N combination locks. Initially, the plan was to use embedded for loops. Within the for loop that cycles through the locks, there would be two more loops. One to increment the original lock(e.g. 1 -> 2 -> 3), and the other to decrement it (9 -> 8 -> 7). There will be a counter in each of these loops to track many times the loop occurred until the desired ending result was obtained. These counters would then be compared and the lower number was used as the final answer for that iteration. Although this method works in theory, there is a major issue with it: its runtime, since it involves embedded *for loops*, making it severely unoptimized.

Deciding to scrap the initial idea, we decided to look into more mathematical strategies in order to code the function. The main strategy we found and implemented was making use of the ***modulo (%) operator***, along with ***division by 10 and the floor() function***. Using % 10 with any number (the start and end lock combinations for our case) isolates the digit in the one's place. This digit can then be used throughout the function. Once the number of turns is found, the original number can then be divided by 10 and put through the floor() function to “take away” the already used digit and put the digit that was previously in the ten’s place into the current one’s place.

Now that the process of cycling through the combination locks is finished, we had to make the algorithm that actually counts the turns needed. Instead of doing the initial idea of incrementing and decrementing, we decided to change our approach and use ***direct distance and wrap-around distance***. The direct distance was obtained by getting the absolute value of the starting value minus the ending value. This is basically the way that the starting value gets to the ending value without incrementing past 9 or decrementing past 0. As for the wrap-around distance, it was obtained by subtracting the obtained value from the direct distance from 10. This subtraction from 10 simulates wrapping around the combination lock, as there are 10 digits total on each lock (0-9). Now that the two distances are obtained, their values are compared and the lower one is used as the number of turns for that specific lock. The number of turns of each lock is added up to get the total number of turns for all locks, thus completing the algorithm.

2. ***Analysis of Algorithm***

Asymptotic analysis was used to analyze the algorithm due the fact that the variable (N) constantly increases. Using this analysis, it can be determined that the runtime increases linearly with N, hence has an O(n) runtime in big o notation. This concept is further explained in the next section (profiling) with test cases.

3. ***Profiling***

In order to create a timer for C language, steps found in the following stack overflow thread were followed.
https://stackoverflow.com/questions/5644730/c-measuring-computing-time

Initially, **1000** loops were measured which resulted in the time printed from the clock function being 0.000000. Hence, we continuously increased the number of loops to run until the times obtained were significant enough to discuss resulting in **5,000,000 loops**.

As mentioned in the specs, two sets of test cases were done. One set with a constant number of locks (4 locks), and another set with an increasing number of locks per testcase (1-5). The results were put into tables below. Three trials were done for each test case in order to obtain more accurate results:

Constant number of locks (time in seconds)

Testcase	Starting Combi.	Ending Combi.	Total turns	Trial 1	Trial 2	Trial 3	Average
1	1048	5146	7	0.12500	0.12800	0.12700	0.126667
2	5203	7937	12	0.13000	0.12600	0.12000	0.125333
3	4728	1615	8	0.12600	0.12500	0.12600	0.125667
4	4352	9120	12	0.12700	0.12700	0.12500	0.12633
5	6767	9696	8	0.12000	0.12800	0.12100	0.123

Min	0.120000
Max	0.130000
Average	0.125400

Increasing number of locks (time in seconds)

Testcase	Starting Combination	Ending Combination	Total turns	Trial 1	Trial 2	Trial 3	Average
1	3	8	5	0.02800	0.02900	0.02900	0.028667
2	43	96	8	0.06400	0.05800	0.06100	0.061
3	954	190	10	0.09100	0.09200	0.09300	0.092
4	6111	3374	12	0.12600	0.12700	0.12900	0.127333
5	17392	84209	11	0.15300	0.16200	0.16000	0.158333

Min	0.028000 (1 digit)
Max	0.162000 (5 digits)
Average	0.093467

Based on the results, it can be seen that increasing the number of locks has a greater impact on the runtime than varying the total number of turns does. The result does make sense, as the amount of time the for loop runs is only affected by the former, not the latter. More locks means more loops being done which ultimately increases the runtime. This idea is also supported with the fact that increasing the number of locks is O(n) in big o notation, while varying the total number of turns is only O(1).

4. ***Assembly and Performance***

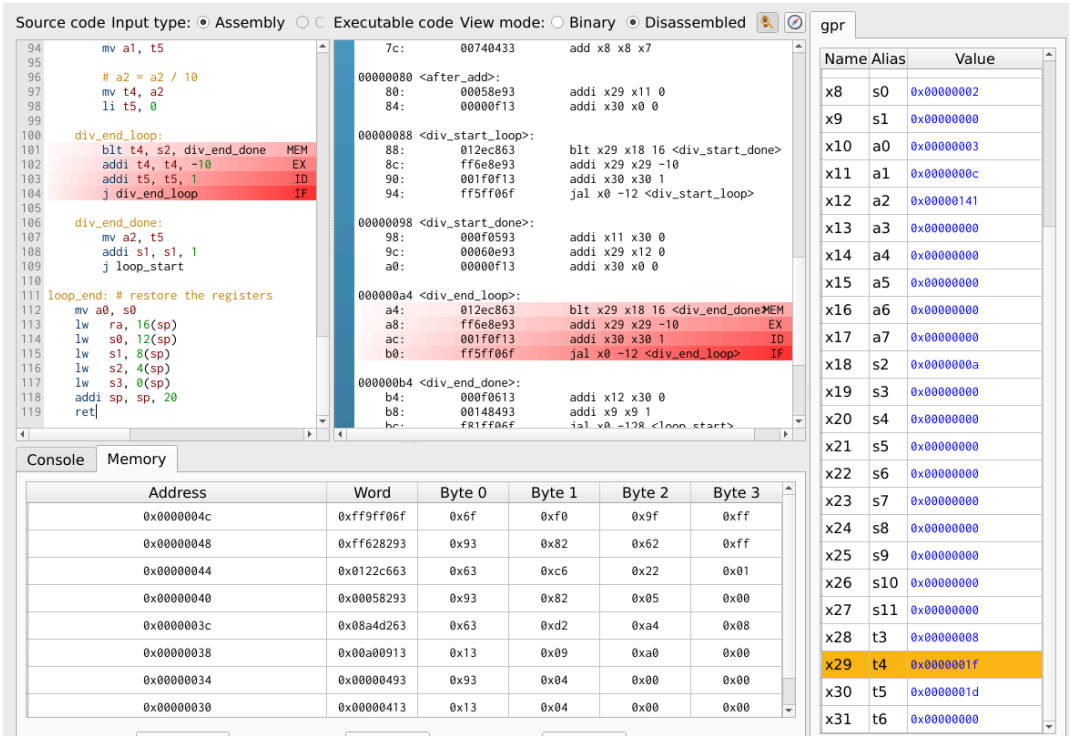
The first iteration of this assembly code contains a lot of loops and branch conditions in order to somewhat replicate the C code that we created. While creating the code, we avoided trying to use complex instructions in order to reduce the complexity of the code (so that we can also feed the pipeline more efficiently).

For this assembly code profile, we used a relatively smaller number of turns and only 3 inputs:

```
main:
    # read num_dials
    li a0, 3
    li a1, 123
    li a2, 321

    jal find_min_combi
```

We also ran our code through RIPES to simulate the feeding of instructions into the 5-stage RISC-V processor:



What we observed from profiling the execution of the program, is that these blocks (image below) are usually where the program repeatedly loops, causing it to be the primary performance bottleneck of the assembly code.

This behavior is expected when using **repeated subtraction blocks to compute for the modulo and division by 10** parts of the C code. This only becomes more of a bottleneck when we consider larger magnitudes that are used, creating more branch evaluations and arithmetic operations.

While writing this however, **<u>loop unrolling</u>** was one of the optimizations that we thought about if we were to implement this again. From its observed behavior, it is expected at the very least that we would need to go through more than 1 run of this block. So by partially unrolling the loop, we can reduce the number of branch and jump instructions that need to be executed. Along with this, we noticed a lot of "***nop***"s that are used to stall the pipeline, maybe due to the flushing from wrong branch predictions.

Another optimization that could have been done as well is **<u>re-ordering the instructions</u>** to let those that have no dependencies come first to improve the pipeline utilization. By scheduling the instructions that are independent of the results of other instructions, then we can also reduce data hazards and improve instruction-level parallelism. Another would be to implement a **<u>re-order buffer</u>** that can be used for **<u>out-of-order</u>** execution (somewhat out of scope for the topics that were discussed up until this point of CoE 163, but I (Marie) learned it before hehe).

Overall, improving pipeline efficiency either through the methods mentioned, can help in order to fill the pipeline with instructions instead of ***nops***, and also minimizing the branch overhead is just one of the many ways to optimize these blocks to accommodate complexity of the modulo operation and the division using repeated subtraction.

```
00000054 <remainder_sub_end>:
    54:         01234663        blt x6 x18 12 <remainder_end_don ID
    58:         ff630313        addi x6 x6 -10                IF
    5c:         ff9ff06f        jal x0 -8 <remainder_sub_end> WB

00000060 <remainder_end_done>:
    60:         406283b3        sub x7 x5 x6
    64:         0003d463        bge x7 x0 8 <abs_val_done>
    68:         407003b3        sub x7 x0 x7
```

```
00000088 <div_start_loop>:
    88:         012ec863        blt x29 x18 16 <div_start_done>
    8c:         ff6e8e93        addi x29 x29 -10              WB
    90:         001f0f13        addi x30 x30 1               MEM
    94:         ff5ff06f        jal x0 -12 <div_start_loop>   EX

00000098 <div_start_done>:
    98:         000f0593        addi x11 x30 0                ID
    9c:         00060e93        addi x29 x12 0                IF
    a0:         00000f13        addi x30 x0 0
```

We also took into account the number of cycles that RIPES simulated, using different processor optimization techniques (or none at all), to highlight the differences between having a processor that utilizes forwarding and hazard detections, as well as a single-cycle processor:

Takes into account the hazards and also utilizes forwarding in the RV32IMC processor:

Cycles:	666
Instrs. retired:	421
CPI:	1.58
IPC:	0.632
Clock rate:	12.66 Hz

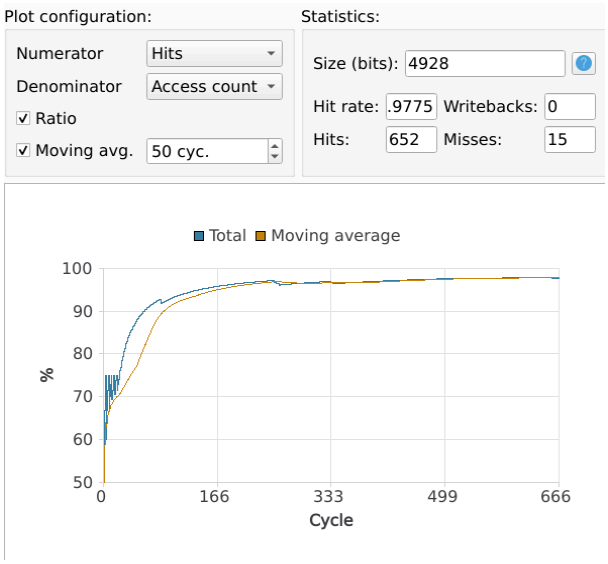
Without forwarding or hazard detection (before we decided to stop simulating):

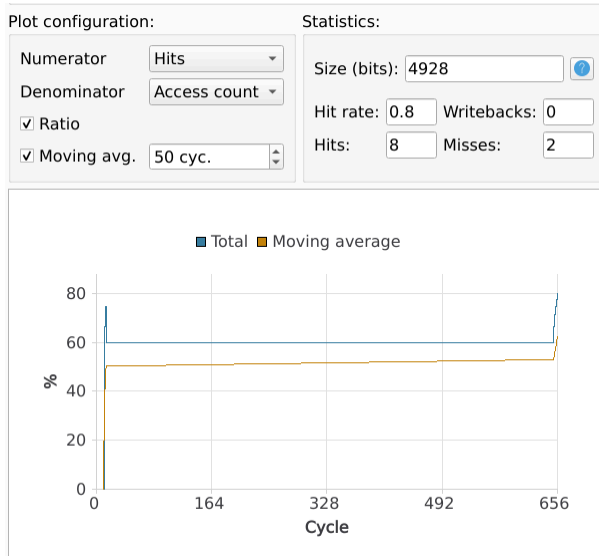
Cycles:	3660158
Instrs. retired:	2321898
CPI:	1.58
IPC:	0.634
Clock rate:	0 Hz

Cycles:	421
Instrs. retired:	421
CPI:	1
IPC:	1
Clock rate:	9.09 Hz

If implemented on a single-cycle processor:

We also noticed in the RIPES simulation that our code had a really good hit rate for the L1 instruction cache (0.9775), and a decent hit rate for the data cache (0.8).





Now when we compiled the C code into https://godbolt.org/ , we noticed that there were some “odd” values used. However from previous discussions, this is probably the most optimized way of implementing this C code into RISC-V assembly.

```
.LBB0_2:
        lw      a0, -16(s0)
        lui     a1, 419430
        addi    a1, a1, 1639
        mulh    a2, a0, a1
        srli    a3, a2, 31
        srai    a2, a2, 2
        add     a3, a3, a2
        li      a2, 10
```

as well as..

```
.LBB0_10:
        lw      a0, -16(s0)
        lui     a1, 419430
        addi    a1, a1, 1639
        mulh    a0, a0, a1
        srli    a2, a0, 31
        srai    a0, a0, 2
        add     a0, a0, a2
```

With further inspection, this appears to be a more reliable way of dividing by 10. Instead of creating the division by repeated subtraction, the compiler gives us a sequence that allows us to roughly divide by 10 through some formula using mulh and srai (so the upper 32 bits that are then multiplied by the number that was loaded using lui), and then a shift to the right. This actually is able to also avoid the use of div, which is costly when it comes to cycle counts, and instead of having some complexity of O(value/10), we instead have it reduced to O(1). (Honestly mindblowing to see how such weird numbers are able to optimize the code)

The branches are also reduced in the optimized version in comparison to ours, and the instructions in our code introduce multiple loops to execute (which is not ideal due to the increase of stalling and hazards, and mispredicted branches). In contrast, the optimized code replaces these loops, containing more arithmetic operations that are beneficial to the overall instruction / cycle count of the processor. This allows for the instruction-level parallelism mentioned earlier to optimize the pipeline, filling it with more useful instructions thus improving the throughput.

Though we would like to note that one key difference between the code we wrote and the code of the compiler is that the code we wrote is able to make use of the cache and exploit localities since we obtained a high hit rate in RIPES because we also accessed a small percentage of the available registers, preserving the spatial and temporal locality (especially for the instruction level cache).

However this comes with the trade off with algorithmic efficiency, as we cannot really predict the runtime of our code due to it being dependent on the value of the inputs for the lock. While on the other hand, the longer, optimized code is better algorithmically but not memory wise but still is great when it comes to improving the throughput of the pipeline.